Ders 13 Genişletilmiş Notu: Co-Citation, Bibliographic Coupling ve PageRank Algoritması

Ders 13 Genişletilmiş Notu: Co-Citation, Bibliographic Coupling ve PageRank Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Konuların Tekrarı: Centrality ve Prestige
- 2. Rank Prestige Formülü ve Özdeğer Sistemi
- 3. Co-Citation ve Bibliographic Coupling
- 4. Web Grafı ve Popülerlik Tabanlı Sıralama
- 5. PageRank'in Temel Fikri
- 6. Oyuncak Örnek: Denklem Sistemi
- 7. Stokastik Komşuluk Matrisi
- 8. Power Iteration Yöntemi
- 9. Rastgele Yürüyüş Yorumu ve Markov Zinciri Teoremi
- 10. Üç Soru ve İki Sorun: Spider Trap ve Dead End
- 11. Spider Trap İçin Çözüm: Teleport
- 12. Dead End İçin Çözüm: Sanal Linkler
- 13. Google Matrisi
- 14. Rank Prestige'den PageRank'e

1. Önceki Konuların Tekrarı: Centrality ve Prestige

Geçen hafta ders yapılamadığı için hoca, ondan önceki haftada kalınan yeri hatırlatarak başladı. Link analizi (link analysis) bölümündeyiz. Bu bölümde önce sosyal ağ analizi (social network analysis) metriklerini gördük ve onları iki başlık altında inceledik: centrality metrikleri ve prestige metrikleri.

Her iki grup da bir sosyal ağ içindeki aktörün önemini belirtir. Fark, önemin hangi linklere bakılarak hesaplandığındadır. Bir düğümün (node) dış dünyaya, yani diğer düğümlere verdiği linklere o düğümün **outlink**'leri, dış dünyadan aldığı linklere ise **inlink**'leri denir.

- **Centrality metrikleri:** Degree centrality, closeness centrality ve betweenness centrality. Outlink'ler üzerinden hesaplanır.
- **Prestige metrikleri:** Degree prestige, proximity prestige ve rank prestige. Inlink'ler üzerinden hesaplanır.

Prestige metriklerini konuşurken önemli bir gözlem yapılmıştı: Alınan her linkin önem derecesi aynı değildir. Önemli bir düğümden link alıyorsanız, o link üzerinden aldığınız önem de daha yüksek olur. Prestige metrikleri içinde bunu yakalayabilen metrik rank prestige'dir.

2. Rank Prestige Formülü ve Özdeğer Sistemi

Ağda n tane düğüm olsun ve i. düğümün rank prestige değerini hesaplamak isteyelim. Bütün düğümlerin rank prestige değerlerini alt alta yazalım. i'ye link verenlerin başına sıfırdan

farklı bir katsayı, link vermeyenlerin başına sıfır koyalım. Böylece link veren düğümlerin rank prestige değerleri toplanıp i'ye yansıtılır.

Katsayı 1 değildir. Hoca bu derste şunu vurguladı: Katsayı öyle seçilir ki düğüm kendi rank prestige değerini outlink sayısı kadar paya bölerek yansıtsın. Bu yüzden link veren bir j düğümü için katsayı $1/O_j$ olur. O_j, j'nin dış dünyaya verdiği link sayısıdır:

$$P_R(i) = \sum_{j=1}^{n} A_{ji}\, P_R(j), \qquad A_{ji} = \begin{cases} \dfrac{1}{O_j} & j,\ i\text{'ye link veriyorsa} \\ 0 & \text{aksi halde} \end{cases}$$

Bu denklemi n düğümün hepsi için alt alta yazarsanız, aslında bir matris-vektör çarpımının açık halini yazmış olursunuz. Solda komşuluk matrisinin (adjacency matrix) transpozesi, onun yanında rank prestige sütun vektörü, sonuç olarak yine bir rank prestige sütun vektörü vardır. Sağ taraftaki vektöre t anındaki, sol taraftaki vektöre $t+1$ anındaki rank prestige vektörü denir:

$$P_{t+1} = A^T P_t$$

Bu denkleme doğrusal cebirde özdeğer sistemi (eigensystem) denir. Çözmek için kullanılan yöntem power iteration'dır. Çözüm sonunda bulunan sütun vektörü, ağdaki bütün düğümlerin rank prestige değerlerini verir. Denklemi çözerken sağlanması gereken bazı koşullar vardır. Bunlar PageRank konusunda ele alınacaktır.

> **Hocanın vurgusu:** Birazdan konuşulacak popülerite tabanlı algoritmalar, yani PageRank ve HITS, tamamen rank prestige metriğinin üzerine kurulmuştur.

3. Co-Citation ve Bibliographic Coupling

PageRank ve HITS'e geçmeden önce hoca, sosyal ağ metriklerinin dışında kalan iki metriği daha anlattı.

Elimizde bir doküman seti olsun ve dokümanların hepsi akademik yayın olsun. Her akademik yayının bir referans bölümü vardır ve başka yayınlara atıf (citation) verir. Size "birbiriyle ilişkili dokümanları bulun" denirse iki yol vardır. Birincisi içeriğe bakmaktır: Dokümanların benzerliğini TF-IDF skoruyla hesaplar ve hangi dokümanların ne oranda benzediğini söylersiniz. İkincisi içeriğe hiç bakmadan yalnız atıflara bakmaktır. Co-citation ve bibliographic coupling bu ikinci yolun metrikleridir.

3.1 Atıf Matrisi

Hoca tahtaya bir matris çizdi. Satırlarda 1'den n'e kadar dokümanlar, sütunlarda yine 1'den n'e kadar dokümanlar vardır. Bir doküman başka bir dokümana atıf veriyorsa ilgili hücreye 1, vermiyorsa 0 yazılır. Yani **satırlar atıfı verenleri, sütunlar atıfı alanları** gösterir. Bu matrise atıf matrisi (citation matrix) denir. Aşağıda bu matris L ile gösterilmiştir: $L_{ij} = 1$ ise i, j'ye atıf vermiştir.

3.2 Co-Citation

Co-citation metriği iki doküman arasındaki ilişkiyi hesaplar. i ve j dokümanları için şöyle düşünülür: Bir k dokümanının referans bölümünde hem i'ye hem j'ye atıf verilmiş olsun. Bu durumda i ve j birlikte atıf almıştır (co-cited). Aralarında bir ilişki olabilir, aynı konuda olabilirler. Çünkü k'nin yazarı oturmuş ikisini aynı makalede kaynak göstermiştir. Bunu yapan makalelerin sayısı arttıkça i ile j arasındaki ilişki de artar.

Matris üzerinde bu, i ve j dokümanlarına ait iki **sütun vektörüne** bakmak demektir. Bir k satırında iki sütunda da 1 varsa, k dokümanı ikisine de atıf vermiştir. Bu durum çok sayıda satırda oluyorsa ilişki güçlü, az sayıda oluyorsa zayıftır. Hesap için iki sütun vektörünün her satırdaki elemanları çarpılır ve çarpımlar toplanır:

$$C_{ij} = \sum_{k=1}^{n} L_{ki} L_{kj}$$

Demek ki iki dokümana aynı anda atıf veren dokümanların sayısı arttıkça co-citation değeri yükselir.

3.3 Bibliographic Coupling

Co-citation'da iki doküman aynı dokümandan atıf alıyordu. Bibliographic coupling'de yön terstir: İki farklı doküman aynı dokümana atıf verir. i, k'ye atıf veriyor ve j de k'ye atıf veriyor. Bu iki doküman çok sayıda ortak dokümana atıf veriyorsa aralarında bir ilişki olduğunu söyleriz.

Hocanın bu metriğin değerini gösteren örneği şudur: Birisi bir makaleyi almış, İngilizceden Fransızcaya çevirmiş olsun. Biri tamamen Fransızca, diğeri tamamen İngilizcedir. Aralarında TF-IDF uygularsanız herhangi bir benzerlik bulamazsınız. Oysa ikisi de aynı makalelere atıf verir, ortak referans sayıları çok yüksektir. Buradan yola çıkarak aralarında bir ilişki olduğunu söyleyebilirsiniz.

Matris üzerinde bu kez iki **satır vektörüne** bakılır. Satır vektörlerinin her elemanı birbiriyle çarpılır ve çarpımlar toplanır. İki vektörde aynı anda 1 olma durumu ne kadar çoksa toplam o kadar yüksek olur:

$$B_{ij} = \sum_{k=1}^{n} L_{ik} L_{jk}$$

> **Hocanın vurgusu:** Bu iki metrik önemlidir. Hoca bunları özellikle matris üzerinde göstermesinin sebebini mantığın akılda kalması olarak açıkladı. Formülü ezberlemeye gerek yoktur. Matris üzerindeki mantığı (co-citation'da sütunlar, bibliographic coupling'de satırlar) hatırladığınızda formülü kolayca yazabilirsiniz.
>
> **Ek bilgi (derste anlatılmadı):** Bu iki toplam aslında matris çarpımının elemanlarıdır: $C = L^T L$ ve $B = LL^T$. Yani C_{ij}, $L^T L$ matrisinin (i, j) elemanı; B_{ij} ise LL^T matrisinin (i, j) elemanıdır. Bütün çiftler için hesap bu çarpımla tek seferde yapılabilir.

4. Web Grafı ve Popülerlik Tabanlı Sıralama

PageRank bizim için önemlidir, çünkü bu derste üzerinde çalıştığımız karmaşık ağ web grafıdır (web graph). Sosyal ağ analizi tekniklerini web grafı üzerinde kullanıyoruz.

Hoca slaytta farklı ağ örnekleri gösterdi: Facebook verisinden oluşan bir sosyal ağ, blogların birbirine verdiği linklerden oluşan bir ağ, bir alandaki makalelerin birbirine verdiği linklerden oluşan bir ağ. Webin alt katmanındaki iletişim ağı olan internet de karmaşık bir ağ oluşturur. Orada sunucular arasındaki fiziksel bağlantılara bakılır. Bizim ilgilendiğimiz ise internetin üzerindeki uygulama sunucularında duran HTML dokümanlarının birbirine verdiği linklerden oluşan web grafıdır. Bu grafta **HTML sayfaları düğüm, köprü bağlantılar (hyperlink) ise linktir**.

4.1 Neden Popülerlik?

Arama (search) olmasaydı web sitelerini tek tek gezerek aramak zorunda kalırdık. Arama yaparken bir anahtar kelimeyi (keyword) sayfalarla eşleştirmeye çalışırız. Bunu TF-IDF ve benzeri metriklerle benzerlik tabanlı yapabilirsiniz. Ama yalnız böyle yaparsanız arama uygulamanız spam'e açık olur. Sayfalar kendileriyle ilgisi olmayan anahtar kelimeleri kullanırsa, ilgisiz ve güvenilir olmayan sayfalar ilk 10 sonuçta görünebilir. Bu yüzden arama sonuçları popülerite tabanlı metriklerle sıralanmak istenir.

Bu sıralama, arama sonuçlarında karşılaşılan iki sorunu çözer:

- **Hangi sayfaya güvenmeliyim?** Diyelim ki "web data mining" diye 3 anahtar kelime girdik ve karşılık 100 sayfa geldi. Bir kısmı üniversite ve Wikipedia sayfalarından, bir kısmı hiç bilinmeyen kişilerin sayfalarından geliyor. Sonuçlarda önce üniversite ve Wikipedia sayfalarını görmek isteriz. Popülerite tabanlı skor bunu sağlar.
- **Sorguya en iyi uyan doküman hangisi?** Sayfalar popülerliklerine göre sıralandığı için, hem anahtar kelimeyle eşleşen hem de popülerliği yüksek sayfa doğrudan ilk sonuç olarak gösterilebilir.

Özetle sayfaları popülerite tabanlı metriklerle sıralamak istiyoruz, çünkü güvenilir ve sorguyla eşleşen sayfaları ilk 10 sonuçta göstermek istiyoruz. Bunu da webin link yapısından (link structure) yararlanarak yapıyoruz.

Slayttaki basit örnekte mavi düğüm çok sayıda düğümden, kırmızı düğüm yalnız bir düğümden link alır. Grafiğe bakınca mavinin daha önemli ve popüler olduğu görülür. Aranan kelime iki sayfada da varsa mavi düğümü mutlaka ilk 10 sonuçta göstermek isteriz. Bunu sağlayan popülerite tabanlı metriklerden biri PageRank, diğeri HITS'tir. Bu derste PageRank işlendi.

5. PageRank'in Temel Fikri

PageRank der ki: Bir düğüme link veren bütün düğümler kendi önem değerlerini bu düğüme aktarır. Çok sayıda düğümden link alan düğüm önemli bir düğümdür. Bu mantık rank prestige'de de vardı.

Slayttaki örnekte Stanford'ın ana sayfası çok sayıda link almıştır, çok kişinin bilmediği bir kişisel sayfa ise yalnız bir link almıştır. Aranan kelime ikisinde de varsa Stanford'ın sayfasını sonuçlarda görmek isteriz.

PageRank'in ikinci söylediği şudur: Link veren her düğümün önem değeri farklıdır. Yüksek PageRank değerli bir düğümden link alıyorsanız, o link yüzünden sizin PageRank değeriniz de yüksek olur.

5.1 Düğüm Büyüklükleri Örneği

Hoca, bir grafta PageRank algoritmasının çalıştırıldığı sonucu gösterdi. Yuvarlakların büyüklüğü PageRank değerini gösterir. Bu slayttan üç düğüm öne çıkar:

- **B düğümü (mavi):** Ağın en önemli düğümüdür. Bunun ana nedeni çok sayıda düğümden link almasıdır.
- **C düğümü:** Hocanın ifadesiyle en az B kadar büyüktür, ama yalnız bir link almıştır. O tek link, ağın en önemli düğümü olan B'den gelir. Çok önemli bir düğümün PageRank değeri C'ye yansır, bu yüzden C'nin değeri de çok yüksek çıkar.
- **E düğümü:** 4 farklı düğümden link almıştır. Yani C'den daha fazla link almıştır. Ama link veren düğümlerin PageRank skorları düşüktür. Bu yüzden E'nin PageRank değeri de düşük çıkar.

Demek ki PageRank hesabında iki şeye bakılır: Düğümün aldığı linkler ve bu linkleri veren düğümlerin PageRank skorları.

5.2 Bir Düğümün PageRank Değeri

Slayttaki örnekte kırmızı işaretli j düğümüne iki düğümden link gelir: i ve k. j'nin PageRank değerini bulmak için i ve k'den gelen değerler toplanır. Ama olduğu gibi değil, outlink sayılarıyla oransal olarak toplanır. i'nin 3 outlink'i vardır, değeri 3'e bölünür. k'nın 4 outlink'i vardır, değeri 4'e bölünür:

$$r_j = \frac{r_i}{3} + \frac{r_k}{4}$$

j'den sonra 3 mavi düğüm vardır. j dış dünyaya 3 link verdiği için her linkten bir sonraki düğüme PageRank değerinin üçte birini, yani $r_j/3$'ü aktarır.

Bu özyinelemeli (recursive) bir algoritmadır. Her düğüm için kendisinden önce gelen düğümlerin verdiği PageRank ve kendisinden sonraki düğümlere verdiği PageRank hesaplanır. Bütün

düğümler için bu belirli sayıda iterasyon tekrarlanır. Bir noktadan sonra değerlerin değişmediği görülür. O zaman en son bulunan değişmeyen değerler, ağdaki düğümlerin PageRank değerleri kabul edilir.

6. Oyuncak Örnek: Denklem Sistemi

Üç düğümlü bir oyuncak graf ele alınır: Y, A ve M. Linkler şöyledir:

- Y kendisine ve A'ya link verir (2 outlink).
- A, Y'ye ve M'ye link verir (2 outlink).
- M yalnız A'ya link verir (1 outlink).

Az önceki anlatımın genel formülü şudur. Herhangi bir j düğümünün PageRank değeri, ona link veren düğümlerin oransal PageRank değerlerinin toplamıdır. Oransal değer, link veren düğümün PageRank değerinin kendi outlink sayısına (d_i) bölünmüş halidir:

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

Hoca bunu Y için adım adım uyguladı. Y'ye link veren iki düğüm vardır: kendisi ve A. Y'nin 2 outlink'i olduğu için $r_y/2$, A'nın 2 outlink'i olduğu için $r_a/2$ alınır. Aynı işlem A ve M için de yapılır (A ve M denklemleri ara adım olarak eklendi):

$$r_y = \frac{r_y}{2} + \frac{r_a}{2}, \qquad r_a = \frac{r_y}{2} + r_m, \qquad r_m = \frac{r_a}{2}$$

3 denklem ve 3 bilinmeyen vardır (r_y, r_a, r_m). Ama bu sistem tek bir sonuç vermez. Bulunan çözümler birbirinin katlarıdır: Bir üçlü sayı çözümse, onun bütün katları da çözümdür. Biz ise r_y, r_a, r_m için tek bir üçlü değer arıyoruz. Bunun için bir denklem daha eklenir: Graftaki bütün PageRank değerlerinin toplamı 1 olsun.

$$r_y + r_a + r_m = 1$$

Artık sistemin tek çözümü vardır. Çözüm şöyle bulunur (ara adım eklendi). İlk denklemden $r_y = r_a$, üçüncü denklemden $r_m = r_a/2$ çıkar. Bunlar toplam denklemine yazılınca $r_a + r_a + r_a/2 = 1$, yani $r_a = 2/5$ bulunur. Sonuç olarak $r_y = 2/5$, $r_a = 2/5$, $r_m = 1/5$ olur. Bu değerler bölüm 8'de power iteration ile bulunan $6/15$, $6/15$, $3/15$ değerlerinin aynısıdır.

> **Ders sonrası not:** Hoca, 3 bilinmeyenli sistemi çözmek için 4 denkleme ihtiyaç olduğunu söyledi; doğrusu sorun denklem sayısı değildir. 3 denklem birbirinden bağımsız değildir: Python ile katsayı matrisi $M - I$'nın rankı 2 bulunmuştur. Bu yüzden çözüm tek değildir. Toplam denklemi eksik olan bağımsız koşulu sağlar.

Denklem setini bu şekilde yazıp çözmek ancak küçük bir grafta mümkündür. Üzerinde çalıştığımız graf web grafı olduğu için aynı işi matris gösterimiyle (matrix representation) yapmak zorundayız.

7. Stokastik Komşuluk Matrisi

Hoca tahtaya web grafının matrisini çizdi. Satırlarda ve sütunlarda 1'den n'e kadar düğümler vardır. Bu matriste **link veren sayfalar sütunlarda, linki alan sayfalar satırlardadır**. Normalde i düğümü j düğümüne link vermişse hücreye 1, vermemişse 0 yazılır. Bu matrise komşuluk matrisi denir, çünkü düğümlerin birbirine verdiği linkleri, yani komşuluklarını gösterir.

Şimdi matrise bir şey daha eklenir: Link varsa hücreye 1 yerine $1/d_i$ yazılır. d_i, i. düğümün dış dünyaya verdiği link sayısıdır. Bu durumda her sütunun elemanları toplanınca 1 elde edilir. i iki sayfaya link vermişse sütununda $1/2$ ve $1/2$ vardır, toplam 1'dir. Üç sayfaya link vermişse $1/3$, $1/3$, $1/3$ vardır, toplam yine 1'dir.

Sütun elemanlarının toplamı 1 olan matrislere **stokastik matris** denir. Sütun toplamları 1 olan komşuluk matrisine de **stokastik komşuluk matrisi (stochastic adjacency matrix)** denir ve M ile gösterilir.

M'nin yanına bir sütun vektörü koyalım. Buna PageRank vektörü diyelim ve r ile gösterelim. $n \times n$ boyutlu matris ile $n \times 1$ boyutlu vektör çarpılınca yine $n \times 1$ boyutlu bir sütun vektörü çıkar:

$$r = M\,r$$

Bu çarpımın neden PageRank formülünü verdiğini görmek için j. düğümün değerine, yani r_j'ye bakalım. r_j, M'nin j. satır vektörü ile r'nin çarpımından çıkar. j. satırda, j'ye link veren düğümlerin hücrelerinde $1/d$ değerleri, diğerlerinde sıfır vardır. r vektöründe ise t anındaki bütün düğümlerin PageRank skorları vardır. Hoca, j'ye üç düğümün link verdiğini düşündürerek gösterdi: Birinci link verenin katsayısı birinci link verenin skoruyla, ikincininki ikincinin skoruyla, üçüncününki üçüncünün skoruyla çarpılır ve bunlar toplanır. Bu toplam r_j'dir. Yani oyuncak örnekteki formülün aynısıdır. Matrisle vektörü çarpmak, aynı formülü bütün düğümler için aynı anda uygulamak demektir.

7.1 Özvektör Problemi

Doğrusal cebirde şu denklem görülür: Bir matris ile bir sütun vektörünün çarpımı, aynı vektörün λ katına eşittir:

$$Ax = \lambda x$$

Burada λ'ya özdeğer (eigenvalue), x'e özvektör (eigenvector) denir. Bizim denklemimiz bunun özel halidir: $1 \cdot r = Mr$, yani $\lambda = 1$. Doğrusal cebirin önerdiği bir teknikle bu denklem çözülürse, bulunan r sütun vektörü aradığımız PageRank skorlarını verir. Bu teknik power iteration yöntemidir.

7.2 Oyuncak Örneğin Matrisi

Y, A, M örneği için matris şöyle kurulur. Sütunlar link verenler, satırlar link alanlardır.

- Y'nin 2 outlink'i vardır: Y'ye ve A'ya. Y sütununda Y ve A satırlarına $1/2$ yazılır.
- A'nın 2 outlink'i vardır: Y'ye ve M'ye. A sütununda Y ve M satırlarına $1/2$ yazılır.
- M'nin 1 outlink'i vardır: A'ya. M sütununda A satırına $1/1 = 1$ yazılır.

$$M = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{pmatrix} \qquad (\text{sıra: } y,\ a,\ m)$$

Her sütunun toplamı 1'dir, yani matris stokastiktir. Hocanın deyimiyle stokastik komşuluk matrisini yazmak bu kadar kolaydır. Geriye yalnız PageRank değerlerini tutan sütun vektörü kalır.

8. Power Iteration Yöntemi

Elimizde stokastik komşuluk matrisi ile sütun vektörünün çarpımından yine sütun vektörü veren bir denklem var. Bunu çözmek için power iteration yöntemi kullanılır. Yöntem kendini tekrar eden bir işlemdir ve şu adımlardan oluşur:

- **1. Başlangıç:** Grafta N düğüm varsa PageRank vektörünün her elemanına $1/N$ atanır. Bu, sıfırıncı andaki PageRank vektörüdür. Slaytta vektör satır gibi yazılmış ama transpozesi alınmıştır, yani aslında sütun vektörüdür: $r^{(0)} = [1/N, \ldots, 1/N]^T$.
- **2. Çarpım:** PageRank vektörü stokastik komşuluk matrisiyle çarpılır ve $t+1$ anındaki vektör bulunur: $r^{(t+1)} = M\,r^{(t)}$.

- **3. Kontrol:** $t+1$ anındaki vektörden t anındaki vektör çıkarılır ve fark önceden belirlenmiş bir ε değeriyle karşılaştırılır. Fark ε'dan küçükse, yani iki vektör birbirine eşit ya da aradaki fark ihmal edilebilir küçüklükteyse durulur. Bulunan vektör nihai PageRank vektörüdür. Değilse 2. adıma dönülür.

  Ders sonrası not: Hoca fark vektörünün ε ile karşılaştırıldığını söyledi; bir vektör tek bir sayıyla doğrudan karşılaştırılamaz. Doğrusu fark vektörünün büyüklüğüne, yani bir normuna bakmaktır: $\|r^{(t+1)} - r^{(t)}\| < \varepsilon$. Slaytta büyük olasılıkla bu norm yazılıdır.

Soru: Başlangıçta neden $1/N$ veriyoruz? Stokastik yapı zaten $1/N$ biçiminde oluşmuyor mu, $r^{(0)}$'a neden tekrar $1/N$ veriyoruz?

Cevap: Stokastik olan M matrisidir, yani M'nin her sütununun toplamı 1'dir. Burada ise r konuşuluyor. Denkleme başlarken her düğüme bir başlangıç PageRank değeri vermek gerekir. Hepsine 1 mi, hepsine 0 mı vereceğiz? Bir değerle başlamak zorundayız ve burada $1/N$ seçildi. Bunu seçmek zorunlu değildir. Birazdan görülecek teori, hangi başlangıç skorlarıyla başlanırsa başlansın aynı PageRank değerine yakınsanacağını söyler.

Ders sonrası not: Hoca soruyu cevaplarken "hepsine sıfır mı vereyim" diye seçenekleri saydı ve teori hangi değerle başlanırsa başlansın aynı sonucu verir dedi; bu ifade başlangıç vektörünün olasılık vektörü olması koşuluyla doğrudur. Python ile Y, A, M örneğinde sıfır vektörü sıfırda kalmış, hepsi 1 olan vektör ise $[1{,}2;\ 1{,}2;\ 0{,}6]$ değerine, yani sonucun 3 katına gitmiştir.

Soru: t anındaki vektörden sıfırıncı anı çıkarınca ε değerini buluyormuşuz gibi anladım. Doğrudan ε'u mu buluyoruz?

Cevap: Hayır. İki vektör arasındaki farkın, önceden belirlenmiş minimum bir değerin altında olup olmadığına bakıyoruz. Aslında baktığımız şey yakınsamadır. İşlemi bir for döngüsü içinde sürekli yapıyoruz. Diyelim ki 50 iterasyon sonunda öyle bir noktaya gelinecek ki bir sonraki PageRank değeri ile bir önceki arasındaki fark neredeyse sıfır olacak. O noktada power iteration durur ve oluşan vektördeki değerler graftaki düğümlerin PageRank değerleri olarak kullanılır.

8.1 Oyuncak Örnekte Power Iteration

Verilen: Y, A, M örneğinin M matrisi ve $N = 3$ olduğu için $r^{(0)} = [1/3,\ 1/3,\ 1/3]^T$. Hoca M ile $r^{(0)}$'ın çarpılmasıyla $r^{(1)}$'in bulunduğunu, farkın ε'dan büyükse $r^{(1)}$ ile devam edildiğini anlattı. Slaytta yalnız bütün iterasyonlar tamamlanınca çıkan sonucu gösterdi. Aşağıdaki ilk adımların değerleri ara adım olarak eklendi ve python ile doğrulandı.

t	r_y	r_a	r_m
0	$1/3$	$1/3$	$1/3$
1	$1/3$	$1/2$	$1/6$
2	$5/12$	$1/3$	$1/4$
3	$3/8$	$11/24$	$1/6$
...	...	...	...
son	$6/15$	$6/15$	$3/15$

Örneğin ilk adımda $r_y = \frac{1}{2}\cdot\frac{1}{3} + \frac{1}{2}\cdot\frac{1}{3} = \frac{1}{3}$, $r_a = \frac{1}{2}\cdot\frac{1}{3} + 1\cdot\frac{1}{3} = \frac{1}{2}$, $r_m = \frac{1}{2}\cdot\frac{1}{3} = \frac{1}{6}$ olur.

Sonuç: Hoca, yakınsama sonunda değerleri Y için $6/15$, A için $6/15$, M için $3/15$ olarak okudu. Python ile 200 iterasyon sonunda $[0{,}4;\ 0{,}4;\ 0{,}2]$ bulunmuştur. Bu, hocanın okuduğu değerlerle aynıdır.

Ek bilgi (derste anlatılmadı): Yöntemin adı, adımların üst üste yazılmasından gelir: $r^{(t)} = M^t r^{(0)}$. Yani aslında matrisin kuvvetleri alınmaktadır. Sütun toplamı 1 olan her matrisin özdeğerlerinden biri 1'dir. Bu yüzden $\lambda = 1$ durumu PageRank'e özgü bir seçim değil, stokastik matrisin bir özelliğidir.

9. Rastgele Yürüyüş Yorumu ve Markov Zinciri Teoremi

Şimdiye kadar PageRank anlatılırken düğümün değerini link verdiği düğümlere oransal olarak aktardığı söylendi. Hoca aynı şeyi bu kez rastgele yürüyüş yorumu (random walk interpretation) ile, yani bir kullanıcının webi dolaşma mantığıyla anlattı.

Bir sayfadasınız ve bir HTML dokümanını inceliyorsunuz. Bir sonraki sayfaya gitmek için sayfadaki hyperlink'lerden birine basarsınız. Şöyle bir tarayıcı (browser) düşünün: Geri düğmesi yok, geri dönemiyorsunuz. URL yazılacak metin kutusu da yok, kendiniz istediğiniz sayfaya gidemiyorsunuz. Yapabileceğiniz tek şey sayfadaki linklere basmaktır.

Sayfada 10 link varsa, birini seçip gitme olasılığınız $1/10$'dur. Slayttaki örnekte i_1 sayfasının 10 linki var ve j sayfasına gitme olasılığı $1/10$'dur.

t anında i_1, i_2 veya i_3 sayfalarında olabilirsiniz, $t+1$ anında j sayfasında olmak istiyorsunuz. Bir rastgele gezginin (random surfer) j'de bulunma olasılığı şöyle hesaplanır: i_1'de bulunma olasılığı çarpı i_1'den j'ye geçme olasılığı $(1/10)$, artı i_2 için aynısı, artı i_3 için aynısı. Genel olarak:

$$p_j(t+1) = \sum_{i \to j} \frac{p_i(t)}{d_i}$$

Yani önceki söylediğimiz "PageRank değerleri oransal aktarılır" cümlesi şimdi olasılık bakış açısıyla söylenmiş oldu. Formül aynıdır. Matris biçiminde de aynıdır:

$$p(t+1) = M\,p(t)$$

$p(t)$, gezginin t anında her düğümde bulunma olasılıklarını gösteren vektördür. M ile çarpılınca $t+1$ anındaki olasılık vektörü çıkar. Denklemi çözerken belirli bir iterasyondan sonra bu vektör bir değere yakınsıyorsa, bu değer **durağan dağılımdır (stationary distribution)**: Elemanları artık değişmez.

Random walk yorumunu anlatmanın sebebi şudur: Random walk teorisinden gelen Markov zinciri (Markov chain) süreçleri teorisi şunu söyler. M matrisi belirli koşulları sağlarsa, denklem çözüldüğünde p'nin yakınsadığı değer her zaman tektir (unique) ve başlangıçta seçilen olasılık değerleri ne olursa olsun aynıdır. Dersin başında "$1/N$ ile başlayalım" denmesinin zorunlu olmamasının dayanağı budur.

> **Hocanın vurgusu:** Bu teorem çok önemlidir. Web grafında milyonlarca düğüm var ve milyonlarca düğüm için bir PageRank vektörü hesaplanıyor. Bunun sonsuz döngüye (infinite loop) girmesi hiç istenmez. Belirli bir iterasyondan sonra bir değere yakınsamalıdır. Teorem, matris belirli koşulları sağlarsa bunun kesinlikle gerçekleşeceğini söyler.

10. Üç Soru ve İki Sorun: Spider Trap ve Dead End

Çıkış noktamız $r_j = \sum_{i \to j} r_i/d_i$ formülü ve bunun matris biçimi $r = Mr$ idi. Şimdi üç soruya cevap vermemiz gerekiyor:

- Bu denklem power iteration ile çözüldüğünde herhangi bir PageRank değerine yakınsar (converge) mı?
- Yakınsıyorsa, yakınsadığı PageRank vektörü bizim istediğimiz şey mi?
- Yakınsadığı PageRank skorları mantıklı mı?

Bu sorulara doğru cevap alabilmek için stokastik komşuluk matrisinin hangi koşullara sahip olması gerektiğine bakılır. Matris bir grafı ifade ettiği için soru şuna döner: Bir grafta karşılaşılabilecek sorunlar nelerdir?

10.1 Spider Trap: İki Düğümlü Örnek

Graf A ve B düğümlerinden oluşsun. A, B'ye; B, A'ya link verir, başka yere gidilemez. Markov zinciri teoremine göre başlangıç değeri ne olursa olsun yakınsama bekliyorduk. Başlangıç vektörü olarak $[1,\ 0]$ seçelim (ilk eleman A, ikinci eleman B):

$$\begin{pmatrix}1\\0\end{pmatrix} \to \begin{pmatrix}0\\1\end{pmatrix} \to \begin{pmatrix}1\\0\end{pmatrix} \to \begin{pmatrix}0\\1\end{pmatrix} \to \dots$$

Bu böyle sürer ve hiçbir değere yakınsamaz. Grafta böyle bir döngü varsa, yani A'dan başlayıp belli adımda B'ye, B'den başlayıp belli adımda yine A'ya gidiliyor ve dışarı çıkılamıyorsa, buna **spider trap (örümcek tuzağı)** sorunu denir.

10.2 Dead End: İki Düğümlü Örnek

Dead end (çıkmaz uç) sorununda bir düğüm link alır ama kimseye link vermez. Oyuncak grafta A, B'ye link veriyor, B kimseye link vermiyor. Başlangıç yine $[1,\ 0]$ olsun:

$$\begin{pmatrix}1\\0\end{pmatrix} \to \begin{pmatrix}0\\1\end{pmatrix} \to \begin{pmatrix}0\\0\end{pmatrix}$$

Yakınsıyor, ama yakınsadığı şey istediğimiz şey değil. Her düğümün PageRank değerinin birbirinden farklı olmasını isteriz ki aralarında sıralama yapabilelim ve hangisinin göreli olarak daha önemli olduğunu söyleyebilelim. Hepsinin değeri birbirine eşit çıkarsa bundan yararlanamayız. Yani yakınsama tek başına yetmez, mantıklı bir değere yakınsamak gerekir.

10.3 Gerçek Web Grafında

Gerçek hayattaki web grafında iki sorun da vardır. Bazı düğümler dışarıdan link alır ama kimseye link vermez (dead end). Bazı alt graflar kendi aralarında link verir ama dış dünyaya vermez (spider trap): İçlerine girdiğinizde sürekli içeride dönersiniz. Böyle bir grafı stokastik komşuluk matrisiyle gösterip power iteration uygularsak, kullanılabilir bir PageRank vektörüne ulaşamayız. Bu yüzden iki sorunun da önce çözülmesi gerekir.

> **Ders sonrası not:** Hoca, dead end ve spider trap varsa Markov zinciri teoremine göre "hiçbir zaman" yakınsanamayacağını söyledi; doğrusu teoremin garantisi bu durumda geçerli değildir. Hocanın kendi örneklerinde de görüldüğü gibi sonuç ya yakınsamamak ya da kullanılamaz bir değere yakınsamaktır.

10.4 Spider Trap: Üç Düğümlü Örnek

Y, A, M grafında bu kez M yalnız kendine link versin. Y, Y'ye ve A'ya; A, Y'ye ve M'ye link verir. A'dan M'ye gelen bir gezgin hep M'ye döner. Bir düğümden başlayıp belirli adımdan sonra hep aynı düğüme dönülüyorsa orada spider trap vardır.

$$M = \begin{pmatrix}1/2 & 1/2 & 0\\ 1/2 & 0 & 0\\ 0 & 1/2 & 1\end{pmatrix}$$

Başlangıç $[1/3,\ 1/3,\ 1/3]^T$ ile power iteration çalıştırılır. Hoca yakınsanan değeri slaytta gösterdi ama sayısını okumadı. Değerler python ile hesaplanmıştır (ara adım eklendi):

t	r_y	r_a	r_m
0	$1/3$	$1/3$	$1/3$
1	$1/3$	$1/6$	$1/2$
2	$1/4$	$1/6$	$7/12$
3	$5/24$	$1/8$	$2/3$
...	...	...	...

t	r_y	r_a	r_m
son	0	0	1

Yakınsadı, ama bütün önem M'de toplandı. Bu, istenen bir PageRank skoru değildir, çünkü ağ üzerinde bir sıralama çıkaramayız. Power iteration burada spider trap yüzünden işe yaramadı.

10.5 Dead End: Üç Düğümlü Örnek

Y, A, M grafında M bu kez kimseye link vermesin. Gezgin dolaşıp M'ye gelir ve oradan diğer düğümlere dönemez. M'nin sütunu tamamen sıfırdır:

$$M = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \end{pmatrix}$$

Sütunlardan birinin toplamı 1 olmadığı için bu matris artık stokastik değildir. Başlangıç $[1/3,\ 1/3,\ 1/3]^T$ ile power iteration çalıştırılırsa (ara değerler eklendi ve python ile doğrulandı):

t	r_y	r_a	r_m
0	$1/3$	$1/3$	$1/3$
1	$1/3$	$1/6$	$1/6$
2	$1/4$	$1/6$	$1/12$
3	$5/24$	$1/8$	$1/12$
...	...	...	...
son	0	0	0

Hocanın okuduğu sonuç da $[0,\ 0,\ 0]$'dır. Bu kullanılabilir bir PageRank vektörü değildir.

11. Spider Trap İçin Çözüm: Teleport

Rastgele gezgin şimdiye kadar yalnız sayfadaki linkleri izleyerek ilerliyordu. PageRank'in önerdiği çözüm gezgine yeni bir yetenek vermektir: Tarayıcıya metin kutusu konur. Gezgin istediği an web grafındaki herhangi bir URL'i yazıp istediği sayfaya geçebilir.

Spider trap'li grafta A'dan M'ye gelen gezgin yalnız M'nin linklerini izleseydi hep M'ye gelirdi. Yeni çözümde M'den istediği anda Y'ye, A'ya veya M'ye zıplayabilir. Bu yeteneğe **teleport** denir. Kural şudur:

- β olasılıkla sayfadaki mevcut linkleri izler.
- $1-\beta$ olasılıkla herhangi bir sayfaya zıplar.

β için kullanılan değer 0,8 ya da 0,9'dur. Hoca genelde %80 alındığını, dolayısıyla $1-\beta$'nın %20 olduğunu söyledi.

Formülü görmek için önce β'yı ve artıdan sonraki kısmı yok sayalım. Geriye alışık olduğumuz formül kalır: j'ye link verenlerin oransal değerleri toplanır. Teleport bunun yanına şunu ekler. Herhangi bir sayfaya atlama olasılığı $1/N$'dir, çünkü N sayfa vardır ve metin kutusuna bunlardan birinin URL'i girilebilir. Linkleri izleyerek j'ye gelme olasılığı ile URL yazarak j'ye gelme olasılığı toplanınca, gezginin $t+1$ anında j'de olma olasılığı çıkar:

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1-\beta)\frac{1}{N}$$

Gezgin bir spider trap'e girmiş olsa bile teleport imkanı sayesinde tekrar grafın geri kalanına dönebilir. Hoca bu değere sıçrama katsayısı (jumping factor) de dedi.

Ders sonrası not: Hoca, spider trap'li örnekte "jumping factor'üm 0,8" ve 0,2 olasılıkla "herhangi bir linki takip ederek geçeyim" dedi; doğrusu 0,8 (β) link izleme olasılığı, 0,2 ($1-\beta$) rastgele herhangi bir sayfaya zıplama olasılığıdır. Zıplama sayfadaki linkleri izlemek değildir. Hoca bunu önceki anlatımında da böyle tanımlamıştır.

Ders sonrası not: Hoca β için 0,8 veya 0,9 değerini verdi; PageRank'in özgün makalesinde sönüm katsayısı (damping factor) büyük olasılıkla 0,85 alınmıştır.

12. Dead End İçin Çözüm: Sanal Linkler

Dead end sorunu da teleport mantığıyla çözülür, ama bu kez matrisin kendisi güncellenir. Dead end oluşturan düğümden, kendisi dahil bütün düğümlere sanal bir link varmış gibi düşünülür ve bu stokastik komşuluk matrisine yansıtılır.

Örnekte dead end olan düğüm M'dir. M'ye 3 sanal link eklenir: kendisine, Y'ye ve A'ya. M'nin sütununa sıfırdan farklı değerler gelir. Değer $1/d$ kuralına göre $1/3$ olur, çünkü 3 sanal link verilmiştir. Böylece matris yeniden stokastik olur:

$$A = \begin{pmatrix} 1/2 & 1/2 & 1/3 \\ 1/2 & 0 & 1/3 \\ 0 & 1/2 & 1/3 \end{pmatrix}$$

12.1 Düzeltmenin Matris Biçimi

Hoca bu güncellemeyi tahtada ayrıca gösterdi. Sorunlu M matrisine aynı boyutta ikinci bir matris eklenir. Bu ikinci matrisin M sütunundaki değerleri $1/3$, diğer bütün elemanları 0'dır:

$$\underbrace{\begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \end{pmatrix}}_{M} + \begin{pmatrix} 0 & 0 & 1/3 \\ 0 & 0 & 1/3 \\ 0 & 0 & 1/3 \end{pmatrix} = A$$

İkinci matris iki vektörün çarpımından üretilir:

- a: Her düğüm için bir eleman içeren vektördür. Düğümün outlink sayısı 0 ise elemanı 1, değilse 0'dır. Örnekte yalnız M'nin outlink'i olmadığı için $a = [0,\ 0,\ 1]$.
- $\frac{1}{N}e$: Bütün elemanları $1/N$ olan vektördür. Örnekte elemanları $1/3$, $1/3$, $1/3$'tür.

$N \times 1$ boyutlu bir vektör ile $1 \times N$ boyutlu bir vektör çarpılınca $N \times N$ boyutlu bir matris çıkar. M'ye eklenecek matris budur:

$$A = M + \frac{1}{N}\, e\, a$$

Burada $\frac{1}{N}e$ sütun vektörü, a ise satır vektörüdür.

Ders sonrası not: Hoca çarpımı $a^T \cdot \frac{1}{N}e$ sırasıyla, yani a'nın transpozesi sütun ve $\frac{1}{N}e$ satır olacak biçimde anlattı; doğrusu tersi sıradır. Python ile kontrol edildiğinde hocanın sırası $1/3$ değerlerini M **satırına** yazar ve matrisi stokastik yapmaz. M **sütununu** dolduran çarpım, sütun olarak $\frac{1}{N}e$ ile satır olarak a'nın çarpımıdır. Hocanın tahtada gösterdiği sonuç matrisi ise doğrudur.

Elde edilen A matrisi hem stokastiktir hem de hocanın tanımıyla indirgenemezdir (irreducible), yani içinde dead end yoktur.

13. Google Matrisi

Markov zinciri teoremine dönelim. Teoremin söylediği şuydu: M belirli koşulları sağlıyorsa, başlangıç vektörü ne olursa olsun, belirli bir iterasyon sonunda power iteration bir PageRank vektörüne yakınsar. Bu değer durağan dağılımdır ve değişmez. Koşullar şunlardır:

- **Stokastik olmalı:** Her sütunun elemanları toplamı 1 olmalı.
- **İndirgenemez (irreducible) olmalı:** Hocanın anlatımına göre içinde dead end sorunu olmamalı.
- **Periyodik olmayan (aperiodic) olmalı:** Hocanın anlatımına göre içinde spider trap sorunu olmamalı.

Dead end için sanal linkler, spider trap için sıçrama katsayısı eklendi. İkisi birlikte son matrisi verir. A matrisindeki linkler β olasılıkla izlenir, $1-\beta$ olasılıkla da herhangi bir sayfaya zıplanır:

$$G = \beta A + (1-\beta)\frac{1}{N}\, e\, e^T$$

Burada $e\, e^T$ bütün elemanları 1 olan $N \times N$ matristir. Google matrisi adı, formülün PageRank'i öneren Google kurucularının makalesinden gelmesindendir.

> **Ders sonrası not:** Hoca önce sanal linklerle elde edilen A matrisine de "Google matrisi" dedi; doğrusu bu ad, teleport da eklenmiş son matris içindir. Hoca da sonraki anlatımında son matrisi "nihai matrisim, Google matrisim" diye adlandırmıştır.

Bu matris stokastiktir, çünkü bütün sütun toplamları 1'dir. Periyodik değildir, çünkü sıçrama katsayısı spider trap'i ortadan kaldırır. İndirgenemezdir, çünkü dead end düğümlerin kolonlarına birbirine eşit $1/N$ değerleri yazılmıştır. Yani Markov zinciri teoreminin istediği matrisin ta kendisidir. Bu matris power iteration'a verilirse başlangıç PageRank değerleri ne seçilirse seçilsin, belirli bir iterasyon sonunda vektör durağan dağılıma yakınsar ve değişmez.

> **Hocanın vurgusu:** PageRank skorları büyük graflarda genellikle yaklaşık 50 iterasyonda nihai değere yakınsar. Ama kullanılan matrisin mutlaka stokastik, periyodik olmayan ve indirgenemez olması sağlanmalıdır.
>
> **Ders sonrası not:** Hoca indirgenemezliği "dead end yok", periyodik olmamayı "spider trap yok" diye eşleştirdi; standart tanımlara göre doğrusu büyük olasılıkla şudur: Dead end doğrudan stokastikliği bozar (hoca bunu kendisi de söyledi). Spider trap ise yapısına göre iki koşuldan birini bozar. A ve B'nin birbirine link verdiği örnek indirgenebilir değildir ama periyodiktir. M'nin yalnız kendine link verdiği örnek ise periyodik değildir ama indirgenebilirdir. Teleport iki koşulu birlikte sağlar.
>
> **Ek bilgi (derste anlatılmadı):** Bir Markov zincirinde her durumdan her duruma sonlu adımda gidilebiliyorsa zincir indirgenemezdir. Bir durumdan kendisine dönüş adım sayılarının en büyük ortak böleni 1 ise o durum periyodik değildir. Teleport ile bütün hücreler sıfırdan büyük olduğu için bir adımda her düğümden her düğüme, bu arada kendisine de gidilebilir. İki koşul bu yüzden birlikte sağlanır.

13.1 Spider Trap'li Oyuncak Örnekte Google Matrisi

Verilen: Bölüm 10.4'teki spider trap'li matris ve hocanın kullandığı $\beta = 0{,}8$. Grafta dead end olmadığı için $A = M$'dir. M 0,8 ile, bütün elemanları $1/3$ olan matris 0,2 ile çarpılıp toplanır:

$$G = 0{,}8 \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{pmatrix} + 0{,}2 \begin{pmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{pmatrix} = \begin{pmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{pmatrix}$$

Her sütunun toplamı $15/15 = 1$'dir. Hoca aynı başlangıç değerlerinden başladığını, daha önce kullanılamaz bir değere giden aynı örneğin artık tek bir PageRank vektörüne yakınsadığını söyledi. Değerleri okumadı. Aşağıdaki değerler ara adım olarak eklendi ve python ile hesaplandı:

t	r_y	r_a	r_m
0	$1/3$	$1/3$	$1/3$
1	$1/3$	$1/5$	$7/15$
2	$7/25$	$1/5$	$13/25$
3	$0{,}2587$	$0{,}1787$	$0{,}5627$
...	...	...	...
son	$7/33 \approx 0{,}2121$	$5/33 \approx 0{,}1515$	$21/33 \approx 0{,}6364$

Python ile $[1,\ 0,\ 0]$ başlangıcından da aynı sonuca ulaşılmıştır. Bütün önem artık M'de toplanmaz. Y ve A da sıfırdan büyük değerler alır ve düğümler sıralanabilir.

14. Rank Prestige'den PageRank'e

Konuşulanlar, Google'ın popülerliği hesaplamak için kullandığı ve 1998'de yayımlanan PageRank algoritmasıdır. Hoca dersin başına dönerek bütünü bağladı.

Sosyal ağ analizinde centrality ve prestige metrikleri vardı. İkisi de aktörün önemini belirtir, prestige metrikleri bunu inlink'lere bakarak yapar. Bunların içinde rank prestige, link verenlerin önem değerine bakar. PageRank temelde bu hesaba dayanır. Rank prestige için özvektör denklemi yazılmış ve değerleri bu denklemle hesaplanmıştı.

Google bu formülden yola çıkarak şunu yaptı: Web grafındaki sorunları (spider trap, dead end) hesaba katacak biçimde formülü bir popülerite hesaplama metriğine dönüştürdü ve buna PageRank metriği dedi. Denklem power iteration ile çözüldüğünde sonuca ulaşılabilmesi için yapılması gereken adımları da önerdi. Nihai Google formülüyle belirli bir adımdan sonra PageRank vektörünün belirli bir değere yakınsayacağı ve değişmeyeceği bilinir.

> **Hocanın vurgusu:** Oyuncak örnekler üzerinde şu metriklerin nasıl hesaplandığını bilmeniz gerekiyor: degree centrality, degree prestige, closeness centrality, proximity prestige, bibliographic coupling, co-citation ve PageRank. Sınavda oyuncak bir graf verilip belirli düğümlerin metrik değerleri sorulacak. PageRank hesabı, belirli bir sıçrama katsayısıyla PageRank ve spider trap olan grafta PageRank hesabı da sorulabilir. Hoca bu dersi mutlaka iyi öğrenmenizi istedi.
>
> **Ders sonrası not:** Hoca toparlarken ve hesaplanacak metrikleri sayarken "proximity centrality" dedi; doğrusu proximity prestige'dir. Dersin başındaki listede centrality metrikleri degree, closeness ve betweenness'tır.

Dersin Toparlaması

Bu derste önce rank prestige'in matris biçimi hatırlatıldı. Ardından akademik yayınlarda içeriğe bakmadan ilişki bulan iki metrik görüldü: Co-citation iki dokümana birlikte atıf verenleri, bibliographic coupling iki dokümanın birlikte atıf verdiklerini sayar. Sonra web grafına geçildi. Yalnız içerik benzerliğine dayalı aramanın spam'e açık olduğu ve bu yüzden popülerite tabanlı sıralamanın gerektiği anlatıldı. PageRank önce oyuncak bir örnekte denklem sistemi, sonra stokastik komşuluk matrisi ve $r = Mr$ özvektör denklemi olarak yazıldı ve power iteration ile çözüldü.

Random walk yorumu, PageRank'in bir rastgele gezginin durağan dağılımı olduğunu gösterdi. Markov zinciri teoremi yakınsamanın ve tekliğin koşullarını verdi. Spider trap ve dead end bu koşulları bozduğu için teleport ve sanal linkler eklendi ve Google matrisine ulaşıldı. Popülerite tabanlı metriklerin diğeri olan HITS bu derste işlenmedi, yalnız adı anıldı.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.